



Mejora tu curso

Entrega final del reto/proyecto

28/9/2026

100 Points Possible

Intento 1



En progreso

SIGUIENTE: Presentar tarea

Agregar comentario

Unlimited Attempts Allowed

17/9/2026

▼ Detalles

Entrega final del reto – Implementación, seguridad, calidad y cierre

Objetivo

Implementar una parte funcional del sistema dando prioridad a la **calidad del código**, la **seguridad**, la **automatización mediante CI/CD** y la **documentación del cierre del proyecto**. La entrega debe demostrar no sólo que el software funciona, sino también que fue probado, analizado y desplegado de forma reproducible.

Importante: no es necesario terminar todo el sistema planteado en el avance. Para esta entrega se evaluará principalmente un **módulo funcional y verificable**, acompañado de evidencia técnica suficiente.

1. Implementación del módulo y seguridad — 20 puntos

Implementen al menos un módulo funcional del sistema. Puede ser, por ejemplo, registro y consulta de personas donantes, usuarios, citas, solicitudes, productos, pagos u otra funcionalidad equivalente de su proyecto.

El módulo deberá incluir:

- Al menos una operación de creación y una de consulta mediante interfaz web, API REST u otro mecanismo claramente demostrable.
- Autenticación con **JSON Web Token (JWT)**.
- Al menos dos roles: **ADMIN** y **USER**, o equivalentes.

- Al menos una operación restringida exclusivamente al rol administrativo y otra disponible para usuarios autenticados.
- Validación de que un usuario sin token o sin el rol requerido no pueda acceder a las operaciones protegidas.
- La llave o secreto usado para firmar JWT **no debe quedar escrito directamente en el código fuente**. Utilicen variables de entorno o secretos del repositorio.
- **NOTA: Para los equipos que están haciendo una aplicación móvil, agregar JWT depende de si se están comunicando con un backend. Si sí lo hacen deben agregar JWT, si no lo hacen deben justificar la ausencia de JWT y en el reporte exponer los mecanismos de autenticación/autorización que están usando.**

Pruebas unitarias

El módulo deberá incluir pruebas unitarias automatizadas y alcanzar una **cobertura mínima de 80 %** sobre el código del módulo implementado.

Herramientas gratuitas sugeridas según su tecnología:

Tecnología	Pruebas	Cobertura
Java	JUnit 5	JaCoCo
JavaScript / TypeScript	Jest	Cobertura integrada de Jest
Python	Pytest	pytest-cov
.NET	xUnit o NUnit	Coverlet

La entrega debe incluir evidencia de la ejecución de las pruebas y del porcentaje de cobertura obtenido.

Para alcanzar el nivel **Altamente competente** de la rúbrica, el módulo debe ser completamente funcional, la autenticación y los roles deben funcionar correctamente y la cobertura debe ser igual o superior al 80 %.

2. Pipeline de CI/CD — 25 puntos

Configuren un pipeline que se ejecute automáticamente al menos cuando se haga **push** o **pull request** hacia la rama principal del repositorio.

El pipeline deberá contener, como mínimo, estas tres etapas lógicas:

1. **Pruebas:** ejecutar automáticamente las pruebas unitarias.
2. **Construcción:** compilar, empaquetar o generar el artefacto desplegable de la aplicación.
3. **Despliegue/Delivery:** desplegar automáticamente la aplicación en un entorno de prueba.

Herramientas de acceso libre recomendadas:

- **GitHub Actions**, especialmente conveniente si el repositorio está alojado en GitHub.
- **GitLab CI/CD**, si el repositorio está alojado en GitLab.

- **Docker y Docker Compose**, opcionales pero recomendables para hacer reproducible el entorno de prueba.

Para el despliegue pueden usar cualquiera de estas alternativas:

- Un servicio que tenga un plan gratuito o académico vigente para desplegar su aplicación.
- Una máquina virtual propia o del equipo.
- Un equipo local configurado como servidor de pruebas mediante un **self-hosted runner**.
- Un contenedor Docker desplegado automáticamente en el entorno de prueba elegido.
- En el caso de las aplicaciones móviles el pipeline puede terminar en la generación del APK.

No se calificará una plataforma de nube específica. Se calificará que el proceso de pruebas, construcción y despliegue esté **automatizado y sea demostrable**.

Evidencia mínima esperada

- Archivo del pipeline dentro del repositorio, por ejemplo `.github/workflows/ci-cd.yml`.
- Captura o enlace a una ejecución exitosa.
- Evidencia de que las pruebas se ejecutaron antes del despliegue.
- URL, captura o instrucciones que permitan verificar el entorno de prueba desplegado.

Para alcanzar el nivel **Altamente competente**, las etapas de pruebas, construcción y despliegue deben ejecutarse correctamente de forma automatizada.

3. Pruebas de seguridad y calidad del código — 25 puntos

En esta parte se utilizarán dos tipos de análisis complementarios:

- **SonarQube**: analiza principalmente el **código fuente** para detectar problemas de calidad, mantenibilidad y seguridad.
- **OWASP ZAP**: analiza la **aplicación mientras está ejecutándose**, enviando y observando peticiones HTTP para detectar posibles problemas de seguridad.

La intención no es obtener un reporte perfecto ni corregir absolutamente todos los hallazgos, sino demostrar que el equipo sabe **ejecutar las herramientas, interpretar sus resultados y corregir problemas relevantes**.

3.1 Análisis de calidad con Sonar

Herramientas que pueden utilizar

Durante el desarrollo pueden seguir utilizando la extensión gratuita **SonarQube for IDE** de Visual Studio Code (antes conocida como SonarLint):

- Extensión oficial: <https://marketplace.visualstudio.com/items?itemName=SonarSource.sonarlint-vscode> (<https://marketplace.visualstudio.com/items?itemName=SonarSource.sonarlint-vscode>)

[itemName=SonarSource.sonarlint-vscode\)](#)

- Documentación: <https://docs.sonarsource.com/sonarqube-for-vs-code/>
(<https://docs.sonarsource.com/sonarqube-for-vs-code/>)

La extensión es útil para encontrar *bugs*, *code smells* y algunos problemas de seguridad directamente mientras programan.

Importante: la extensión de VS Code sí cuenta como una herramienta válida de apoyo durante el desarrollo, pero **por sí sola no sustituye el análisis final solicitado en esta entrega**. Para documentar métricas globales del proyecto, como *code smells*, deuda técnica, duplicación o cobertura, deberán ejecutar además un análisis con **SonarQube Community Build** o una alternativa equivalente que genere un tablero de métricas del proyecto.

Opción recomendada: SonarQube Community Build en Docker

SonarQube Community Build puede ejecutarse gratuitamente de forma local. Si ya tienen Docker Desktop instalado, una forma sencilla de iniciar una instancia para fines académicos es:

```
docker run -d --name sonarqube \
-e SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true \
-p 9000:9000 \
sonarqube:latest
```

Después:

1. Esperen a que el contenedor termine de iniciar.
2. Abran en el navegador:

<http://localhost:9000>

3. Inicien sesión. En una instalación nueva las credenciales iniciales suelen ser:

usuario: admin
contraseña: admin

4. Creen un proyecto o permitan que SonarQube lo cree al recibir el primer análisis.
5. Generen un **token de análisis** desde SonarQube.
6. Ejecuten el scanner correspondiente a la tecnología de su proyecto.

Documentación oficial para iniciar SonarQube Community Build con Docker:

<https://docs.sonarsource.com/sonarqube-community-build/server-installation/from-docker-image/installation-overview> (<https://docs.sonarsource.com/sonarqube-community-build/server-installation/from-docker-image/installation-overview>)

Ejemplo para proyectos Java con Maven

Desde la carpeta que contiene el `pom.xml` pueden ejecutar primero sus pruebas y posteriormente el análisis de SonarQube.

Una forma recomendada es guardar el token en una variable de entorno llamada `SONAR_TOKEN` en lugar de escribirlo directamente en el repositorio.

Después pueden ejecutar:

```
mvn clean verify
mvn org.sonarsource.scanner.maven:sonar-maven-plugin:sonar
```

Si su configuración requiere indicar explícitamente la instancia local:

```
mvn org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
-Dsonar.host.url=http://localhost:9000
```

No suban el token de SonarQube al repositorio. Si lo utilizan dentro de GitHub Actions o GitLab CI/CD, guárdenlo como secreto del repositorio.

Para otros lenguajes pueden utilizar el scanner correspondiente de SonarQube para Gradle, NPM, .NET, Python o SonarScanner CLI.

Consideración para proyectos Android

Si su proyecto es una aplicación Android, pueden utilizar **SonarQube for IDE dentro de Android Studio** durante el desarrollo y ejecutar el análisis global mediante **Gradle + SonarScanner** sobre el proyecto. Para la cobertura de pruebas pueden utilizar **JaCoCo** o **Kover**, siempre que el reporte generado pueda integrarse o documentarse junto con el análisis de SonarQube.

En este caso, la evidencia esperada es la misma que para cualquier otro proyecto: tablero general, métricas principales, hallazgos revisados y evidencia de correcciones.

Relación con la extensión de Visual Studio Code

Una forma recomendada de trabajo es:

```
SonarQube for IDE en VS Code
↓
Detectar y corregir problemas mientras se programa
↓
SonarQube Community Build
↓
Analizar el proyecto completo y obtener métricas
↓
Documentar resultados y corregir hallazgos relevantes
```

Si lo desean, pueden configurar **Connected Mode** para conectar la extensión de VS Code con su instancia local de SonarQube. Esto es opcional para la entrega.

Métricas que deben documentar

Registren las métricas que su análisis muestre, especialmente cuando estén disponibles:

- Bugs.
- Vulnerabilities.
- Security Hotspots.

- Code Smells.
- Technical Debt / deuda técnica.
- Duplicated Lines / duplicación.
- Coverage / cobertura, si se integró el reporte de pruebas.

No es necesario que todas las métricas tengan valor cero. Lo importante es que el equipo pueda explicar qué significan y qué acciones tomó ante los hallazgos relevantes.

Evidencia mínima de Sonar

Incluyan:

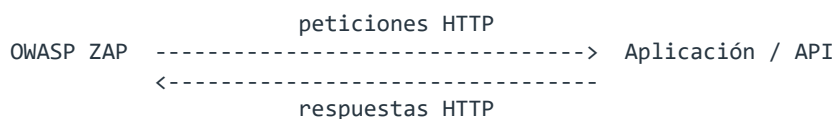
1. Una captura del tablero general del proyecto.
2. Las métricas principales obtenidas.
3. Al menos un hallazgo seleccionado para revisión.
4. Explicación breve de por qué el hallazgo era relevante o por qué se consideró un falso positivo.
5. Evidencia de la corrección cuando corresponda.

3.2 Pruebas de seguridad con OWASP ZAP

OWASP ZAP es una herramienta gratuita y de código abierto para realizar pruebas dinámicas de seguridad sobre aplicaciones web y APIs.

A diferencia de SonarQube, ZAP no necesita analizar directamente el código fuente: trabaja contra una aplicación que ya está ejecutándose.

Ejemplo:



ZAP analiza esas peticiones y respuestas y puede advertir sobre problemas como:

- posibles inyecciones SQL;
- posibles vulnerabilidades XSS;
- encabezados HTTP de seguridad ausentes;
- exposición innecesaria de información;
- cookies configuradas de forma insegura;
- respuestas o endpoints con comportamientos que requieren revisión.

Documentación y descarga oficial:

<https://www.zaproxy.org/download/> (<https://www.zaproxy.org/download/>)

Dos tipos de análisis que deben distinguir

Passive Scan

ZAP observa las peticiones y respuestas que pasan por la herramienta y genera alertas sin intentar explotar activamente la aplicación. Es apropiado como primer análisis y tiene un riesgo bajo para un entorno de prueba.

Ejemplo de hallazgo:

Content-Security-Policy Header Not Set

Active Scan

ZAP envía diferentes entradas y payloads para comprobar si la aplicación puede tener vulnerabilidades. Este análisis puede modificar datos o generar muchas peticiones.

Ejecuten un Active Scan **únicamente contra su propia aplicación o un entorno de prueba para el que tengan autorización**. No utilicen esta práctica contra páginas o APIs de terceros.

Ruta recomendada para esta actividad: aplicación de escritorio

Para esta entrega, la forma más sencilla es utilizar la aplicación de escritorio de ZAP.

1. Inicien su aplicación o API.

Ejemplo:

http://localhost:8080

2. Abran OWASP ZAP.
3. Seleccionen **Quick Start**.
4. Seleccionen **Automated Scan**.
5. Escriban la URL de su aplicación.
6. Ejecuten el análisis.
7. Esperen a que ZAP recorra la aplicación y termine de generar alertas.
8. Revisen la sección **Alerts**.
9. Seleccionen al menos un hallazgo relevante y lean:
 - nivel de riesgo;
 - descripción;
 - URL o endpoint afectado;
 - evidencia mostrada por ZAP;
 - recomendación de solución.
10. Determinen si el hallazgo representa un problema real en su proyecto o un posible falso positivo.
11. Corrijan al menos un hallazgo relevante cuando sea técnicamente razonable.

12. Ejecuten nuevamente el análisis y comparen los resultados.

Una salida típica puede contener alertas agrupadas aproximadamente así:

```
Alerts
├── High
├── Medium
├── Low
└── Informational
```

No se calificará el proyecto por tener necesariamente **cero alertas**. Se calificará la capacidad de ejecutar el análisis, comprender los hallazgos y actuar sobre ellos.

Alternativa con Docker

Quienes ya utilicen Docker pueden ejecutar un **Baseline Scan**, que realiza rastreo y análisis pasivo sin efectuar un ataque activo completo.

Ejemplo:

```
docker run -t ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t http://host.docker.internal:8080
```

En Windows y macOS, `host.docker.internal` permite normalmente que el contenedor acceda a una aplicación que se está ejecutando en la máquina anfitriona.

Para generar un reporte HTML pueden montar una carpeta de trabajo siguiendo la documentación oficial de ZAP:

<https://www.zaproxy.org/docs/docker/baseline-scan/>

(<https://www.zaproxy.org/docs/docker/baseline-scan/>)

Quienes tengan una API documentada mediante **OpenAPI/Swagger** también pueden investigar el **ZAP API Scan**, diseñado específicamente para APIs:

<https://www.zaproxy.org/docs/docker/api-scan/> (<https://www.zaproxy.org/docs/docker/api-scan/>)

Esta alternativa es opcional; para cumplir la actividad es suficiente utilizar correctamente el Automated Scan de la aplicación de escritorio.

Consideración sobre JWT

Si su aplicación utiliza JWT, es posible que algunos endpoints protegidos respondan `401 Unauthorized` durante un escaneo automático. Configurar ZAP para autenticarse automáticamente con JWT es posible, pero requiere configuración adicional y **no es obligatorio para esta actividad**.

Para esta entrega pueden:

- escanear primero los endpoints accesibles sin autenticación;
- revisar manualmente el comportamiento de endpoints protegidos;
- documentar que determinados endpoints no fueron explorados automáticamente debido a la autenticación;

- de manera opcional, configurar autenticación en ZAP si el equipo desea profundizar.

El hecho de que un endpoint requiera autenticación no debe considerarse por sí mismo un error del escaneo.

Evidencia mínima de OWASP ZAP

Incluyan:

1. Captura o reporte del primer escaneo.
2. Una tabla con los hallazgos revisados, por ejemplo:

Hallazgo	Riesgo reportado	Endpoint	Interpretación del equipo	Acción realizada
Falta un header de seguridad	Bajo	<code>/api/...</code>	Configuración incompleta	Se agregó el header
Posible SQL Injection	Alto	<code>/api/search</code>	Se confirmó concatenación de SQL	Se utilizaron parámetros preparados
Alerta no aplicable	Medio	<code>/api/...</code>	Falso positivo	Se documentó la justificación

3. Evidencia del cambio realizado en el código o configuración.
4. Segundo escaneo, cuando sea posible, para mostrar el resultado después de la corrección.
5. Breve conclusión sobre los principales riesgos encontrados.

No basta con adjuntar automáticamente todos los resultados de ZAP. El equipo debe demostrar que **revisó e interpretó** al menos los hallazgos más importantes.

3.3 Consideraciones para aplicaciones móviles Android

Los equipos que desarrollen una aplicación móvil Android deberán distinguir entre **calidad del código**, **seguridad propia de la aplicación móvil** y **seguridad de las comunicaciones con el backend**.

Una forma recomendada de organizar el análisis es:



Sonar en Android

Para Android, Sonar sigue utilizándose para analizar la **calidad del código fuente**. Pueden trabajar con:

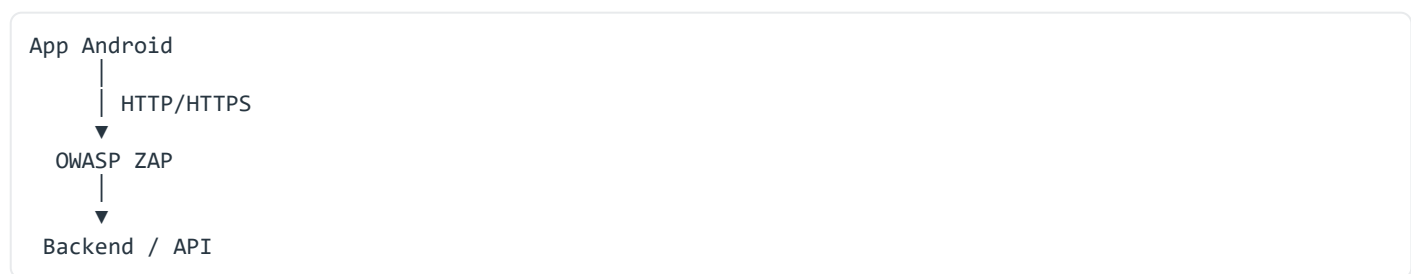
- **SonarQube for IDE** dentro de Android Studio durante el desarrollo.
- **SonarScanner for Gradle** para ejecutar el análisis completo del proyecto.
- **JaCoCo** o **Kover** para generar evidencia de cobertura de pruebas.

No es obligatorio que todos los hallazgos sean corregidos, pero sí deben interpretar los más relevantes y documentar las acciones tomadas.

OWASP ZAP en aplicaciones móviles

OWASP ZAP **no analiza directamente el APK**. Su función en una aplicación móvil es observar y probar el tráfico HTTP/HTTPS entre la app y el backend o servicios externos.

El flujo esperado es:



Para realizar la prueba pueden:

1. Ejecutar ZAP en la computadora.
2. Configurar ZAP para escuchar en un puerto, por ejemplo **8080**.
3. Configurar el emulador Android o dispositivo de prueba para utilizar la computadora como **proxy HTTP/HTTPS**.
4. Abrir la aplicación y recorrer sus funciones principales.
5. Verificar en ZAP las peticiones realizadas por la app, por ejemplo:

```
POST /login
GET /users/123
POST /donations
PUT /profile
```

6. Revisar primero las alertas producidas por el **Passive Scan**.
7. Ejecutar un **Active Scan** únicamente contra una API propia o un entorno de pruebas autorizado.
8. Documentar los hallazgos y, cuando sea posible, corregir al menos uno y volver a probar.

Consideración sobre HTTPS

Cuando la aplicación utiliza HTTPS, ZAP necesita actuar como intermediario entre la app y el servidor. Para ello puede ser necesario instalar o confiar temporalmente en el certificado de ZAP

dentro del emulador o entorno de pruebas.

En Android moderno, una aplicación puede no confiar automáticamente en certificados instalados por el usuario. Para una **build de desarrollo o debug** puede configurarse un `network_security_config` que permita realizar la prueba sin afectar la configuración de producción.

Si la aplicación implementa **certificate pinning**, la interceptación con ZAP puede requerir cambios adicionales en la versión de pruebas. Para esta actividad **no será obligatorio deshabilitar o modificar certificate pinning**; el equipo deberá simplemente documentar esta limitación si aplica.

Herramienta complementaria para seguridad móvil: MobSF

Para analizar problemas propios de Android, pueden utilizar de forma complementaria **MobSF (Mobile Security Framework)**, una herramienta gratuita y de código abierto orientada al análisis de aplicaciones móviles.

MobSF puede ayudar a identificar, entre otros aspectos:

- permisos excesivos;
- componentes exportados incorrectamente;
- secretos o claves embebidos en el código o APK;
- configuraciones inseguras en `AndroidManifest.xml`;
- almacenamiento local de información sensible;
- configuraciones de red que requieren revisión.

MobSF es **opcional** cuando la aplicación Android consume una API y el equipo puede realizar el análisis de tráfico con ZAP.

Sin embargo, si la aplicación **no utiliza backend ni comunicaciones HTTP/HTTPS**, OWASP ZAP no aporta evidencia relevante. En ese caso, el equipo puede utilizar **MobSF como sustituto del análisis con ZAP**, siempre que explique brevemente por qué ZAP no aplica a la arquitectura de su proyecto y documente los hallazgos obtenidos con MobSF.

Evidencia mínima para un proyecto Android

Los equipos con aplicación móvil deberán entregar, según corresponda:

- capturas o reporte de SonarQube;
- evidencia de cobertura con JaCoCo o Kover;
- capturas o reporte de ZAP mostrando tráfico de la aplicación hacia su backend;
- al menos un hallazgo de seguridad interpretado por el equipo;
- evidencia de una corrección o justificación cuando no sea posible corregirla;
- opcionalmente, reporte de MobSF;
- si no utilizan ZAP, una justificación técnica breve y el reporte de MobSF como evidencia alternativa.

En aplicaciones móviles, no se evaluará el uso mecánico de una herramienta específica. Se evaluará que el equipo utilice una herramienta adecuada a la arquitectura de su aplicación y pueda **identificar, interpretar y corregir problemas de seguridad reales**.

3.4 Evidencia conjunta y criterio de evaluación

Como evidencia final de esta sección deberán entregar:

```
reportes/
├── seguridad-zap/
│   ├── reporte o capturas del escaneo inicial
│   ├── evidencia de hallazgos revisados
│   └── evidencia posterior a las correcciones
├── seguridad-movil/           # sólo si aplica
│   └── reporte o capturas de MobSF
└── sonar/
    ├── captura del dashboard
    ├── métricas principales
    └── evidencia de hallazgos revisados/corregidos
```

Para alcanzar el nivel **Altamente competente**, la rúbrica requiere pruebas de seguridad con **identificación y corrección de vulnerabilidades**, así como un análisis completo de calidad del código con las métricas documentadas.

Por tanto, la evidencia ideal sigue este ciclo:

Analizar → interpretar → corregir → volver a analizar → documentar

No se espera que una herramienta determine por sí sola si el proyecto es seguro o de buena calidad. Los resultados de SonarQube y OWASP ZAP deben ser interpretados por el equipo.

4. Cierre del proyecto y análisis — 15 puntos

Elaboren un informe de cierre donde comparen lo que se planeó en el avance del reto con lo que realmente se implementó.

El análisis debe incluir una tabla como la siguiente:

Elemento	Planeado	Ejecutado	Diferencia / causa
Módulo principal	Registro y consulta de donantes	Registro terminado, consulta parcial	Se priorizó autenticación y pruebas
Duración	5 días	7 días	Ajustes en configuración del pipeline
Pruebas	80 % de cobertura	84 %	Cumplido
Despliegue	Día 4	Día 6	Problemas con variables de entorno

Incluyan también las **lecciones aprendidas**, enfocándose en aspectos concretos de ingeniería de software, por ejemplo:

- decisiones de arquitectura que funcionaron o no;

- problemas encontrados al implementar autenticación o roles;
- dificultad para automatizar pruebas o despliegues;
- utilidad de las pruebas unitarias;
- defectos detectados por análisis estático o pruebas de seguridad;
- cambios que harían si comenzaran nuevamente el proyecto.

Para alcanzar el nivel **Altamente competente**, la comparación entre la planificación y la ejecución debe ser detallada y las lecciones aprendidas deben estar claramente explicadas.

5. Plan de mejora continua — 15 puntos

Propongan acciones concretas para continuar el proyecto después de esta entrega.

El plan debe contener acciones **específicas y medibles**. Eviten propuestas demasiado generales como “mejorar la seguridad” o “agregar inteligencia artificial” sin indicar qué se haría exactamente.

Ejemplo:

Mejora propuesta	Acción concreta	Indicador de éxito	Prioridad
Mejorar seguridad	Agregar expiración y refresh tokens	Ningún access token válido por más de 15 min	Alta
Mejorar calidad	Eliminar los 5 code smells críticos reportados por SonarQube	0 code smells críticos	Media
Mejorar CI/CD	Bloquear despliegue si fallan pruebas	100 % de despliegues pasan primero por pruebas	Alta
Innovación	Generar predicción de demanda usando datos históricos	Prototipo funcional con datos de prueba	Baja

El plan deberá incluir por lo menos una propuesta de **innovación**, pero ésta debe estar relacionada con las necesidades reales del proyecto.

Para alcanzar el nivel **Altamente competente**, el plan debe incluir acciones específicas, medibles y propuestas de innovación.

Entregables

Entreguen un archivo ZIP con la siguiente estructura sugerida:

```
entrega-final-equipo-X/  
├── codigo-fuente/  
│   └── proyecto completo o enlace al repositorio  
├── pipeline/  
│   └── archivo(s) YAML del pipeline CI/CD
```

```
├── reportes/
│   ├── pruebas-unitarias/ # es la cobertura
│   ├── seguridad-zap/
│   ├── seguridad-movil/      # sólo si aplica
│   └── sonar/
├── evidencias/
│   ├── pipeline-exitoso.png
│   ├── despliegue.png
│   └── otras-capturas o video/
├── informe-cierre.pdf # puntos 3 y 4 de la rubrica
└── README.md
```

El repositorio debe contener

- Código fuente completo del módulo implementado.
- Pruebas unitarias.
- Configuración de JWT y roles.
- Archivo del pipeline CI/CD.
- Archivos necesarios para construir y ejecutar el proyecto.
- `README.md` con instrucciones claras para:
 - instalar dependencias;
 - configurar variables de entorno;
 - ejecutar la aplicación;
 - ejecutar las pruebas;
 - ejecutar o consultar los análisis de seguridad y calidad.

No incluyan contraseñas, tokens, llaves privadas ni secretos reales dentro del ZIP o del repositorio. Utilicen valores de ejemplo en archivos como `.env.example`.

Reportes técnicos

Incluyan los reportes o evidencias generados por:

- cobertura pruebas unitarias;
- OWASP ZAP, cuando aplique;
- MobSF, si se utilizó para un proyecto móvil;
- SonarQube;
- ejecución del pipeline CI/CD.

Los reportes pueden entregarse en HTML, PDF, XML, capturas de pantalla o el formato nativo de la herramienta, siempre que sea posible verificar claramente los resultados.

Informe de cierre (puntos 4 y 5 de la rubrica)

Entreguen un documento en **PDF o Word**

Relación directa con la rúbrica

Recomendaciones finales

- ▼ **Ver rúbrica**

<https://cursos.tecmilenio.mx/courses/246704/assignments/4467437>

Criterios	Calificaciones			Puntos
1. Implementación del módulo y seguridad.	Altamente competente Desarrolla un módulo completamente funcional con un sistema de autenticación completo usando JWT y roles. Implementa pruebas unitarias exhaustivas con cobertura mayor o igual a 80 %. 20 a >17 pts	Competente Desarrolla un módulo funcional con un sistema de autenticación, pero con problemas menores de implementación. Cobertura de pruebas unitarias entre 60-79 %. 17 a >14 pts	Aún sin desarrollar la competencia Desarrolla un módulo incompleto, con problemas graves de autenticación o roles faltantes. Cobertura de pruebas unitarias por debajo del 60 %. 14 a >0 pts	/20 pts
2. Implementación de pipeline CI/CD.	-- Implementa un pipeline CI/CD completo con etapas automatizadas de pruebas, construcción y despliegue que funcionan correctamente. 25 a >22 pts	-- Implementa un pipeline CI/CD con la mayoría de las etapas requeridas, pero con problemas menores de configuración. 22 a >19 pts	-- Implementa un pipeline CI/CD incompleto que carece de etapas esenciales o con problemas graves de configuración. 19 a >0 pts	/25 pts
3. Pruebas de seguridad y análisis de calidad de código.	-- Realiza pruebas de seguridad exhaustivas con identificación y corrección de vulnerabilidades. Análisis completo de calidad de código con todas las métricas documentadas. 25 a >22 pts	-- Realiza pruebas de seguridad con alguna identificación de vulnerabilidades. Análisis de calidad de código con algunas de las métricas documentadas. 22 a >19 pts	-- No realiza pruebas de seguridad. El análisis de calidad de código está incompleto. 19 a >0 pts	/25 pts

Criterios	Calificaciones			Puntos
4. Cierre del proyecto y análisis.	-- Crea un informe de cierre exhaustivo con comparación detallada de líneas de tiempo planificadas vs. reales y lecciones aprendidas completas. 15 a >12 pts	-- Crea un informe de cierre con la mayoría de los elementos requeridos, pero con falta la comparación o las lecciones aprendidas. 12 a >9 pts	-- Crea un informe de cierre incompleto que carece de componentes esenciales o análisis. 9 a >0 pts	/15 pts
5. Plan de mejora continua.	-- Desarrolla un plan de mejora detallado con acciones específicas y medibles, asimismo genera propuestas de innovación. 15 a >12 pts	-- Desarrolla un plan de mejora con algunas acciones, pero carece de criterios de medición o elementos de innovación. 12 a >9 pts	-- Desarrolla un plan de mejora incompleto que carece de acciones específicas o relevancia para el proyecto. 9 a >0 pts	/15 pts

Elegir un tipo de presentación

T
Texto


URL web


Cargar


Más

Presentar tarea